

BRIGHT - Graph Neural Networks in Real-Time Fraud Detection

Mingxuan Lu
Shanghai Jiao Tong University
mingxuan.lu@sjtu.edu.cn

Zhichao Han
eBay Inc.
zhihan@ebay.com

Susie Xi Rao
Swiss Federal Institute of Technology
in Zurich (ETHZ)
raox@inf.ethz.ch

Zitao Zhang
eBay Inc.
zitzhang@ebay.com

Yang Zhao
eBay Inc.
yzhao5@ebay.com

Yinan Shan
eBay Inc.
yshan@ebay.com

Ramesh Raghunathan
eBay Inc.
raraghunathan@ebay.com

Ce Zhang
Swiss Federal Institute of Technology
in Zurich (ETHZ)
ce.zhang@inf.ethz.ch

Jiawei Jiang
School of Computer Science,
Wuhan University
jiawei.jiang@whu.edu.cn

ABSTRACT

Detecting fraudulent transactions is an essential component to control risk in e-commerce marketplaces. Apart from rule-based and machine learning filters that are already deployed in production, we want to enable efficient real-time inference with graph neural networks (GNNs), which is useful to catch multihop risk propagation in a transaction graph. However, two challenges arise in the implementation of GNNs in production. First, future information in a dynamic graph should not be considered in message passing to predict the past. Second, the latency of graph query and GNN model inference is usually up to hundreds of milliseconds, which is costly for some critical online services. To tackle these challenges, we propose a Batch and Real-time Inception Graph Topology (BRIGHT) framework to conduct an end-to-end GNN learning that allows efficient online real-time inference.

BRIGHT framework consists of a graph transformation module (Two-Stage Directed Graph) and a corresponding GNN architecture (Lambda Neural Network). The Two-Stage Directed Graph guarantees that the information passed through neighbors is only from the historical payment transactions. It consists of two subgraphs representing historical relationships and real-time links, respectively. The Lambda Neural Network decouples inference into two stages: batch inference of entity embeddings and real-time inference of transaction prediction. Our experiments show that BRIGHT outperforms the baseline models by >2% in average w.r.t. precision. Furthermore, BRIGHT is computationally efficient for real-time fraud detection. Regarding end-to-end performance (including neighbor query and inference), BRIGHT can reduce the P99 latency by >75%. For the inference stage, our speedup is on average 7.8× compared to the traditional GNN.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.
CIKM '22, October 17–21, 2022, Atlanta, GA, USA

© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9236-5/22/10...\$15.00
<https://doi.org/10.1145/3511808.3557136>

CCS CONCEPTS

• **Computing methodologies** → **Neural networks**; • **Information systems** → **Enterprise applications**; • **Theory of computation** → **Dynamic graph algorithms**.

KEYWORDS

graph neural networks, fraud detection, heterogeneous graph, dynamic graph, graph inference

ACM Reference Format:

Mingxuan Lu, Zhichao Han, Susie Xi Rao, Zitao Zhang, Yang Zhao, Yinan Shan, Ramesh Raghunathan, Ce Zhang, and Jiawei Jiang. 2022. BRIGHT - Graph Neural Networks in Real-Time Fraud Detection. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM '22)*, October 17–21, 2022, Atlanta, GA, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3511808.3557136>

1 INTRODUCTION

Fraudulent transaction is one of the most serious threats to online financial services today. This problem is exacerbated by the growing sophistication of business transactions using online payment applications (e.g., eBay and Alipay) and payment cards [13, 26]. Fraudsters use a range of tactics, including paying with stolen credit cards, chargeback fraud involving complicit cardholders, selling fake e-gift cards, or creating schemes to create fraud ring attacks against multiple merchants¹. In this work, our goal is to detect risky transactions on a real-world e-commerce platform.

In e-commerce marketplaces, unauthenticated transactions are a major part of buyer risk. In our previous generation of detection engine, it is observed that the *entities* linked to transactions, such as shipping addresses and device machine IDs, are key clues in detecting frauds. Based on these entities, hundreds of patterns are summarized as features in machine learning models or as rules in decision engines. However, the feature engineering of these linkage patterns, which are designed by human experts, is only feasible within 1-hop horizon in the graph. Beyond 1-hop, it is still quite inefficient for human experts to explore and interpret, due to the

¹<https://www.verifi.com/in-the-news/need-know-fraud-rings/> (last accessed: April 22, 2022).

huge amount of derived features generated by aggregating original features along risk propagation paths.

Characteristics of Transaction Graphs. Two vital characteristics of fraudulent transactions have been raised [18] in a similar application of massive suspicious registration detection. First, transactions and relevant entities naturally form a graph with heterogeneous nodes (e.g., IP address, email, accounts). Those fraudulent transactions tend to share certain common risky entities, such as the device and IP address. Second, fraud detection in transaction graphs should consider the problem of temporal dynamic because the accounts used by fraudsters and legitimate users usually generate distinctive activity events in separate time periods.

Challenge 1: Prevention of learning from future transactions. *In a temporally dynamic graph, can we use information flowing from vertices whose timestamps are different from the current vertex?* For the problem of fraud detection, claim, chargeback, and suspension histories are usually considered top-ranking features in fraud detection models, in which timestamp is an important property. For a specific time, if timestamp is considered, there are two categories of events, historical and future events. Intuitively, the feature patterns of the upcoming events linked by entities may allow models to foresee the risk. However, this assumption is impractical in real-time applications, as future vertices have not appeared in the graph yet. Therefore, in this work, we strictly constrain the model to utilize only the past transaction.

Challenge 2: Efficiency of Graph Inference. During graph inference, different financial applications entail diverse latency requirements. Some use cases, such as Know-Your-Customer (KYC) procedures during account registration and post-transaction evaluation for anti-money laundry detection, may tolerate relatively high latency. However, detecting fraud transactions requires a quick response, since end users are often sensitive to latency. Generally, graph neural network (GNN) models aggregate information from neighbors within at least 2-hops in the graph; therefore, each inference typically takes hundreds of milliseconds or even several seconds, which cannot meet internal requirements and user-side expectation on system latency.

To address the aforementioned challenges, we propose the BRIGHT (Batch and Real-time Inception Graph Topology) framework.

- (1) We propose a novel graph transformation method for temporally dynamic transaction graphs, yielding a Two-Stage Directed (TD) Graph storage strategy. This construction is time-sensitive, and can be utilized to perform time-aware message passing in real-time.
- (2) Based on the transformed TD graph, we introduce a Lambda² Neural Network (LNN) architecture that supports both batch processing and real-time processing. LNN leverages snapshot aggregation and avoids foreseeing future information during training. We have also improved the efficiency of real-time inference using the Lambda architecture.
- (3) Our experiments show that BRIGHT outperforms the baseline models by more than 2% w.r.t. average precision. Additionally, it is computationally efficient for real-time fraud

detection. BRIGHT reduces more than 75% P99³ end-to-end inference latency (including neighbor query and graph inference), compared to traditional GNN inference frameworks. For the inference stage, our speedup is on average 7.8× over the traditional GNN.

2 PRELIMINARIES

In our previous works, DHGReg [18] and DyHGN [16], we have studied the dynamics of graph modeling in account and transaction networks.⁴ Specifically, DHGReg [18] is designed to detect massive suspicious registrations through a dynamic heterogeneous graph neural network. DHGReg is composed of two types of subgraphs: (1) structural subgraphs in each timestamp that reflect the links between different types of entity; and (2) a temporal subgraph that links the structural subgraphs via temporal edges. As such, the time dimensions are unrolled in a large graph together with structural edges. With this graph construction technique, DHGReg outperforms simple baselines such as GCN and GAT in capturing suspicious registrations on time.

DyHGN [16] is built on top of DHGReg [16] and has studied to incorporate diachronic temporal embeddings [6] while still keeping structural subgraphs. It also explores the replacement of simple convolution layers with more complex ones, such as a heterogeneous transformer layer [10]. We also discuss graph structure and data distribution over time, as well as their influence on GNN modeling dynamics.

However, we notice three issues when deploying these prototype models in production:

- **(I1)** The bi-graph structure (transactions and entities) tends to deplete GPU memory when the graph scales up to millions of vertices and edges;
- **(I2)** There is future information leakage in the training of predictive models. In the GNN message passing, features or embeddings from future events are also included when evaluating the risk of past events.
- **(I3)** The latency of the neighbor query is more than hundreds of milliseconds or even seconds, which is not ideal for a real-time responsive system.

3 RESEARCH QUESTION AND METHODOLOGY

In this work, we propose BRIGHT (Two-Stage Directed Graph transformation + Lambda Neural Network) to tackle these remaining issues when deploying GNN in production.

- To solve **(I1)**, we keep the structural subgraphs of DHGReg and partition the graphs to support both batch and online learning.
- To solve **(I2)**, we strictly control that only historical reference transaction information is used to predict target transactions.

²Lambda architecture is a hybrid data processing approach that handles a large amount of data both in batch and in streaming fashion. For an overview on Lambda architecture in data processing, we refer readers to https://en.wikipedia.org/wiki/Lambda_architecture (last accessed: April 22, 2022).

³The P99 (99th percentile) latency is the worst latency that was observed by 99% of all requests.

⁴We have discussed fraud detection with heterogeneous GNNs in an industrial billion-scale dataset, and conducted human evaluations against GNN predictions in xFraud [17]. There, we did not include the discussion of temporal aspects.

- To reduce the overhead of querying neighbors in (I3), we enable batch inference and store entity embeddings in a key-value store. These entity embeddings are then used in online inference to decrease inference latency.

In this section, we introduce the research questions we want to answer with BRIGHT (Sec. 3.1), illustrate the graph transformation module (Sec. 3.2), and explain the Lambda Neural Network (Sec. 3.3).

3.1 Research Question

With BRIGHT, our goal is to answer these two research questions.

- (Q1) How could we construct a dynamic graph that allows us to control the direction of the information flow and effectively support online inference?
- (Q2) How could we design a graph neural network architecture that is efficient for online inference?

3.2 Time-sensitive Graph Construction

In this work, transaction fraud detection is treated as a binary classification problem in an inductive setting on a dynamic heterogeneous graph. In a static transaction graph $\mathcal{G}$ (Fig. 1 (a)), a vertex $v \in \mathcal{V}$ has a type $\tau(v) \in \mathcal{A}$, where $\mathcal{A} := \{\text{transaction}, \text{entity}\}$. An edge $e \in \mathcal{E}$ links from a *transaction* vertex (*txn* for short hereafter) to an *entity* vertex.

Nodes *txn* with unauthenticated chargeback claims from the customer system are marked as 1, which are considered fraudulent transactions. The others are marked as 0, which represents legitimate transactions. These labels are used in our binary classification task. These *txn* with labels are called **target** transactions in our setting. The historical transactions that share the same entities *entity* used in the target transaction, such as emails and shipping addresses, we call them **reference** transactions, which do not have any labels in our experiments.

To construct a Two-Stage Directed Graph (TD Graph) to support Lambda Neural Network (LNN), our graph construction consists of the following steps, as illustrated in Fig. 1.

- (a) **Static Graph.** The static graph is constructed from months of transaction logs by transforming them into a bi-graph (transaction-entity graph). Note that the edges here are bidirectional.
- (b) **Snapshot Graph.** Each node in the static graph is placed in its corresponding timestamp snapshot. Reference transactions are linked to entities that are on the same timestamp snapshot as the corresponding target transactions.
- (c) **Two-Stage Directed Graph.** A TD Graph is stored in target transaction partitions. A partition is made up of several timestamps that are the temporal snapshots in which all related transactions and entities are. A TD graph is equivalent to not only a snapshot graph, but also a simplified topology view for target transactions in the partition. The TD Graph is used for LNN learning and inference.

We now discuss in detail how the three types of graphs are constructed in Fig. 1.

3.2.1 Static Graph Construction. To collect neighbor features to assess the risks of transaction fraud, multiple entities used in the checkout sessions are considered neighbors of *txn* nodes. These

Table 1: Notations.

Notation	Description
$\mathcal{G}$	Static graph
$\mathcal{V}$	Vertices on the static graph
$\mathcal{E}$	Edges on static graph
v	Transaction or entity on the static graph
e	Transaction-entity linkage on the static graph
txn	Transaction vertex on the static graph
$entity$	Entity vertex on the static graph
$\mathcal{T}$	Timestamp set/Time window in a partition, $\mathcal{T} := \{0, 1, \dots, t\}$
$\mathcal{G}_{\mathcal{T}}$	Snapshot graph where each vertex has a timestamp
$\mathcal{G}_T^{RT}$	Real-Time (RT) Graph, used in real-time inference
$\mathcal{G}_T^{Batch}$	Batch Graph for batch inference
$\mathcal{G}_T^{TD}$	Two-Stage Directed Graph, union of $\mathcal{G}_T^{RT}$ and $\mathcal{G}_T^{Batch}$
t	Time period snapshot t , a time duration (day)
$\mathcal{G}_t^{RT}$	Subgraph of Real-Time (RT) Graph on $partition_t$
$\mathcal{G}_t^{Batch}$	Subgraph of Batch Graph on $partition_t$
$snapshot_t$	Time period snapshot $t \in \mathcal{T}$
$partition_t$	Time period partition $t \in \mathcal{T}$, which is the partition of target transaction at time t that also contains reference transaction from historical time snapshots
X_t	Feature embeddings of transactions on $partition_t$
txn_t	Transaction on $snapshot_t$
$entity_t$	Entity nodes on $snapshot_t$
txn_t^{tgt}	Target transaction on $partition_t$
txn_t^{ref}	Historical reference transaction on $partition_t$
$entity_t^{tgt}$	Entity nodes on $partition_t$

entities, including shipping address, E-mail, IP address, device ID, contact phone, payment token, and user account, are represented as *entity* nodes in $\mathcal{G}$.

Each *txn* vertex represents a checkout transaction along with a unique transaction ID, linked with multiple *entity* vertices such as shipping addresses, E-mails, contact phones that buyers need to confirm on checkout pages. Most of the *entity* vertices are also linked to multiple *txn* vertices.

Given a set of target transactions, a static graph $\mathcal{G}$ can be constructed from their records. A transaction record could be broken down into a *txn* vertex and several *entity* vertices. Edges e are placed between entities and transactions that use these entities. Reference transactions within a given observation window are extracted if they share the same entities. Edges are also added between reference transactions and common entities.

3.2.2 Snapshot Graph Construction. A time snapshot $t \in \mathcal{T}$, where $\mathcal{T} := \{0, 1, \dots, t\}$, represents a period of time duration, e.g., hour or day. $\mathcal{T}$ is the time window in one Snapshot Graph. In our experiments, the time period for each snapshot is one day. A

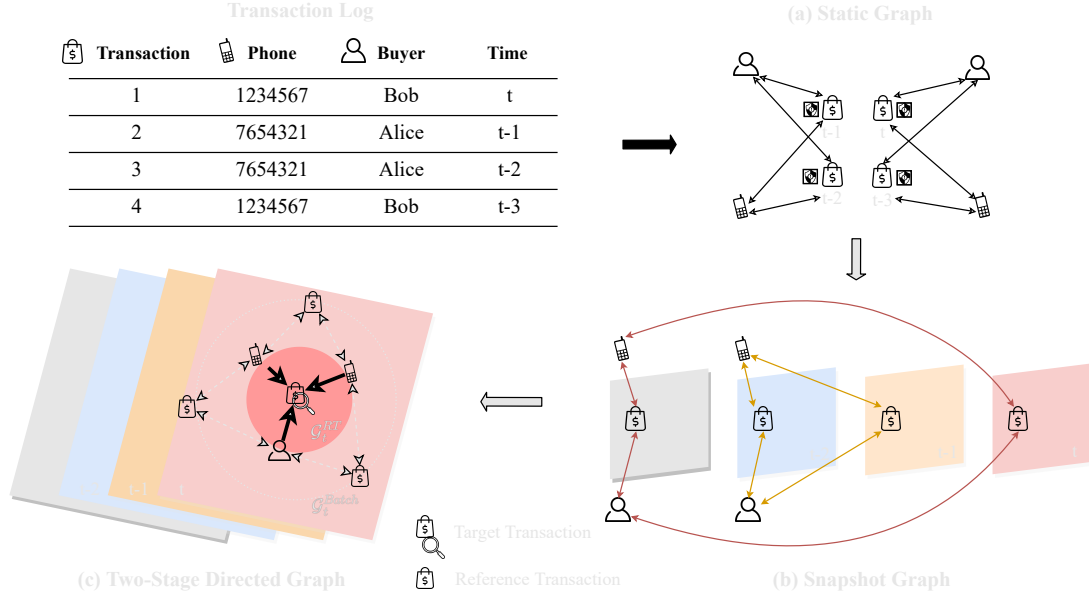


Figure 1: Graph Transformation in BRIGHT. RT: Real-Time.

snapshot vertex $v_t \in \mathcal{V}_{\mathcal{T}}$ represents the static vertex. The snapshot vertex v_t is a transformation of v at snapshot t . The workflow to construct a snapshot graph is described below.

- (1) For each *txn* node on the static graph $\mathcal{G}$, we construct txn_t on *snapshot_t*, which is the time period in which the transaction was created.
- (2) For each *entity* node directly linked to the transaction nodes txn_t , we create the node $entity_t$ on *snapshot_t*, sharing the same transaction creation time. There may be multiple instances of the same entity in the snapshot graph $\mathcal{G}_{\mathcal{T}}$ if it is linked to transactions created in various snapshots.
- (3) Create edges between the target transaction nodes txn_t and the 1-hop entity neighbor $entity_t$.
- (4) Create edges between the reference transaction nodes txn_i and 1-hop entity neighbor $entity_t$, where $0 \leq i \leq t$.

On Snapshot Graph $\mathcal{G}_{\mathcal{T}}$, a transaction node may have two roles. One is the target transaction; the other is the reference transaction if it shares the same entity as another target transaction. In Fig. 1 (b), the transaction in snapshot $t-3$ is a reference transaction to the transaction in snapshot t . Meanwhile, it can also be a target transaction in snapshot $t-3$ when we predict its label.

Potential roles for a single txn_t node make it difficult to isolate future information from message passing if there is no further filtering or sampling based on transaction timestamps. Some edges can be in $\mathcal{G}_{\mathcal{T}}^{RT}$ and $\mathcal{G}_{\mathcal{T}}^{Batch}$ (which we will introduce in Sec. 3.2.3), which leads to future information leakage. We want to avoid neighbor sampling due to the high latency in existing SAGE neighbor sampling approaches, as shown in [30]. This brings us to our design of Two-Stage Directed Graph shown in the next section.

3.2.3 Two-Stage Directed Graph. Two-Stage Directed (TD) graph simplifies the topology view from the target transaction side, making it easy to split the graph into Real-Time Graph $\mathcal{G}_{\mathcal{T}}^{RT}$ and Batch

Graph $\mathcal{G}_{\mathcal{T}}^{Batch}$. We show in Fig. 1 (c) these two parts of the graphs that are taken later by LNN as input for various parts of the network. $\mathcal{G}_{\mathcal{T}}^{RT}$ is the subgraph with thick directed edges (the dark red circle), while $\mathcal{G}_{\mathcal{T}}^{Batch}$ is the subgraph with bidirectional edges (the light red circle).

The partition of target transaction is introduced to isolate transaction roles, because a transaction can act as both a target/reference transaction, as we discuss above. All reference transaction nodes sharing common entities are stored with the target transactions in the same partition. On the TD graph, there is only one role for transactions, either a target transaction or a reference transaction. There are multiple snapshots *snapshot_i* in one target partition *partition_t*, $i \leq t$, as illustrated in Fig. 2 (a). The workflow to construct a TD graph is described below.

- (1) Place target transactions txn_t , their corresponding 1-hop entities $entity_t$, and the reference transactions txn_i , where $0 \leq i \leq t$ into *partition_t*. These transactions are represented as txn_t^{tgt} and txn_t^{ref} . The entities are represented as $entity_t^{tgt}$. Reference transactions may be put into multiple partitions if their related target transactions are in different time snapshots.
- (2) Link txn_t^{ref} and $entity_t^{tgt}$ with bi-directed edges, which forms the Batch Graph $\mathcal{G}_{\mathcal{T}}^{Batch}$. It is used for batch inference of entity representations.
- (3) Link $entity_t^{tgt}$ and txn_t^{tgt} with directed edges from the entity to the target transaction, which forms the Real-Time Graph $\mathcal{G}_{\mathcal{T}}^{RT}$. It is used for real-time inference of transaction risk.

The edges on Batch Graph are bi-directed, which enables bi-directional inference with deep GNN models on Batch Graph. The edge types for the TD Graph are represented in Tab. 2. The edges on Real-Time Graph are uni-directed, which are always from entities

Table 2: Two-Stage Directed Graph Edge Types.

Edge Type	Description
$txn_t^{ref} \leftrightarrow entity_t^{tgt}$	Bi-directed edges between reference transactions and entities on the Batch Graph G_t^{Batch}
$entity_t^{tgt} \rightarrow txn_t^{tgt}$	Directed edges from entities to target transaction nodes on the Real-Time Graph G_t^{RT}

to the target transactions. The number of layers in Real-Time Net is set to 1 for inference efficiency, so it is not necessary to make the edges bi-directed. Furthermore, with this setting, we ensure that no message is passed between target transactions across multiple timestamps even if they are on the same target partition.

Note that the design of the TD graph is in line with the Lambda Neural Network, which enables batch and online inference in the same framework. We elaborate the design of the LNN architecture in the following Sec. 3.3. In Fig. 2 (a), we show an illustrative example of the risk assessment of the transaction on txn_t^{tgt} using TD Graph. The features of historical reference transactions txn_t^{ref} are sent to $entity_t^{tgt}$ (phone and buyer in this example). They are the entity nodes 1-hop away from the target transaction. After bi-directed message-passing through the dotted edges, the embeddings (for phone and buyer in this example) of 1-hop neighbors are obtained. Then these embeddings are sent to the target transaction txn_t^{tgt} through the directed edges (in red), which forms the graph used in real-time inference. With transformation into a TD Graph, the messages passed only from historical entities/transactions prior to txn_t^{tgt} are guaranteed, which addresses the research question (Q1).

3.3 Network Architecture

We propose a Lambda Neural Network (LNN) architecture, which treats the last hop edges in Real-Time Graph differently from those edges in Batch Graph from TD. This is illustrated in Fig. 2 (b).

Lambda Architecture is a hybrid data processing approach that handles a large amount of data in both batch and streaming fashion. One of the benefits from Lambda Architecture is the balance between the data processing scalability and the data access latency. LNN architecture handles message passing from Batch Graph and Real-Time Graph in separate GNN blocks. Message passing on Batch Graph is conducted in batch jobs through Batch Net. Message passing on the Real-Time Graph is conducted in real-time inference through the Real-Time (RT) Net, deployed in online services.

Similar to the two-“tower” model architecture [5, 24, 25, 27–29] (see Sec. 6), we design LNN to support two stages of processing (batch and real-time inference). The LNN architecture, together with the TD graph, addresses the research question (Q2), which is our proposed solution for online inference in a dynamic setting.

As illustrated in Fig. 2, LNN consists of a Batch Net (in blue) and a Real-Time (RT) Net (in orange). Batch Net is a stack of GNN layers, similar to DeepGCNs [14]. RT Net consists of a graph convolutional layer and a decoder, which could simply be a fully connected linear layer. The two stages are divided at the neighboring entities in one

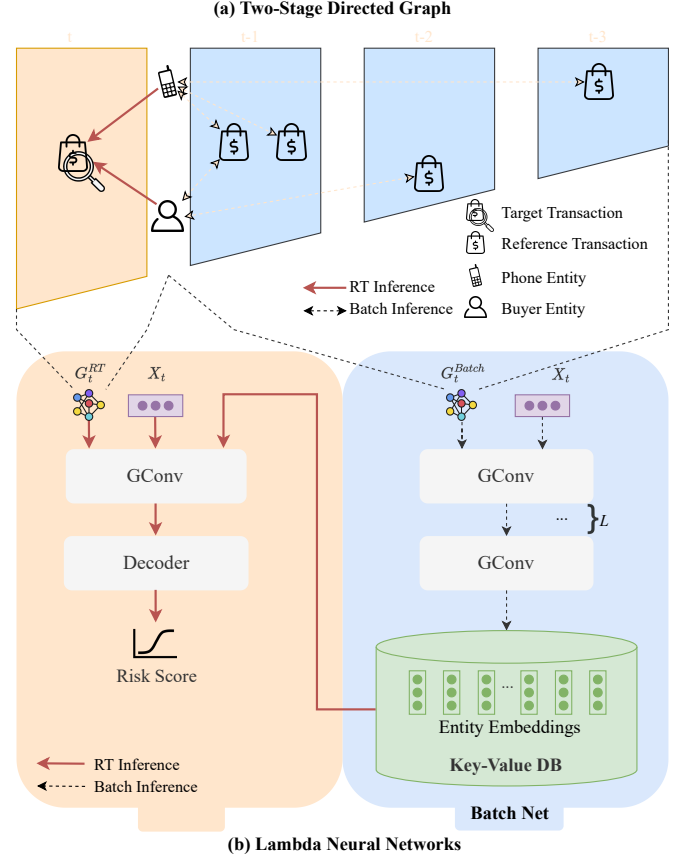


Figure 2: Two-Stage Directed Graph (a) and Lambda Neural Network (b). RT: Real-Time, GConv: Graph convolution.

hop to $entity_t^{tgt}$ (phone and buyer in this example). The first stage is Batch Net inference, after which we obtain the entity embeddings learned from historical transactions and flush them into a key-value store. In the second stage, the RT Net fetches the entity embeddings from the key-value store, also takes the raw features from the target transactions, and then computes the transaction risk scores.

3.3.1 End-to-end Training. In the training phase, both the Batch Graph and the RT Graph are used for end-to-end LNN training. In each mini-batch, we sample one partition for the time window $\mathcal{T}$, where $\mathcal{T} := \{0, 1, \dots, t\}$. The timestamp of each target transaction is within the partition time window. All timestamps for reference transactions are prior to those for target transactions.

Similar to traditional GNNs, the inputs of Batch Net include the features of reference transactions X_t and Batch Graph G_t^{Batch} , as illustrated in Eq. 1:

$$h_t^{Batch} = \text{BatchNet}(X_t, G_t^{Batch}) \quad (1)$$

Each graph convolutional layer of Batch Net is generic with a hidden state h^l of the layer l , as illustrated in Eq. 2. $\mathcal{N}(v)$ is all the neighbors of the vertex v , which is txn_t^{ref} or $entity_t^{tgt}$. $\odot$ is a

generalized node aggregation function. $\cup$ represents concatenation, W weight matrices.

$$h_v^{l+1} = \sigma(W \cdot \odot(\{h_v^l\} \cup \{h_u^l, \forall u \in \mathcal{N}(v)\})) + h_v^l \quad (2)$$

As input, RT Net takes features from target transactions and entity embeddings learned from Batch Net inference. The graph convolutional layer of RT Net shares the same form as that of Batch Net, as is illustrated in Eq. 3. One difference is that there is only one convolutional layer in the RT Net for the sake of inference efficiency.⁵ Furthermore, RT Net isolates the aggregation of messages between different target transaction nodes txn_t^{tgt} within the same target snapshot partition. ϕ is a linear projection for the transaction features, which assures that the transaction features and the embeddings have the identical dimensions.

$$h_{txn_t^{tgt}}^{RT} = \sigma(W \cdot \odot(\{\phi(X_{txn_t^{tgt}})\} \cup \{h_{entity_t^{tgt}}^{Batch}, \forall entity_t^{tgt} \in \mathcal{N}(txn_t^{tgt})\})) + \phi(X_{txn_t^{tgt}}) \quad (3)$$

The risk score for target transaction txn_t^{tgt} is decoded from the real-time transaction embedding $h_{txn_t^{tgt}}^{RT}$ through a fully connected linear layer, which is illustrated in Eq. 4.

$$score_{txn_t^{tgt}} = W \cdot h_{txn_t^{tgt}}^{RT} \quad (4)$$

3.3.2 Efficient Graph Inference. As described above, end-to-end learning uses the complete LNN architecture. When deployed in a production environment, LNN inference is decoupled into batch inference and real-time inference.

(Batch Inference Stage.) For batch inference stage, the embeddings h_t^{Batch} of entities $entity_t^{tgt}$ are periodically generated according to Eq. 1. In our current design, the time period is one day. Finally, entity embeddings are generated and stored in a distributed key-value database for multiple downstream purposes. In our paper, these embeddings are used to detect fraud transactions. The embeddings are refreshed daily to serve real-time transaction risk evaluation.

(Real-Time Inference Stage.) For real-time risk assessment, the second stage of LNN (namely RT Net) calculates the score by Eq. 4. The results are equivalent to end-to-end inference with the complete LNN adopted in the learning stage. The inference latency is further significantly reduced due to the 1-layer RT Graph. Since RT Graph only requires 1-hop neighbors, which are already available in the transaction request, a graph query is not necessary during real-time inference. Graph query refers to fetching nodes and their features from a graph database such as Neo4j. Illustrative queries in Cypher⁶ are given in Tab. 3. For a single layer RT Net, the number of hops is set to 2. It is possible to include more hops in the RT Net, as we have discussed in Sec. 5.3. In the experiments

Table 3: Cypher Queries for Different Number of Hops (in the Extended Graph) in the RT Net. RT: Real-Time.

Number of Hops	Cypher Query
2	MATCH (target:transaction)-[]->(:entity)->[]->(nbr2:transaction) WHERE transaction=\$transaction_id RETURN nbr2
4	MATCH (target:transaction)-[]->(:entity)->[]->(nbr2:transaction) -[]->(:entity)->[]->(nbr4:transaction) WHERE transaction=\$transaction_id RETURN nbr2, nbr4
6	MATCH (target:transaction)-[]->(:entity)->[]->(nbr2:transaction) -[]->(:entity)->[]->(nbr4:transaction) -[]->(:entity)->[]->(nbr6:transaction) WHERE transaction=\$transaction_id RETURN nbr2, nbr4, nbr6

Table 4: Dataset Summary ("M": Million). *The fraud ratios are only reported on the sampled datasets. TD: Two-Stage.

Split	Graph	#Nodes	#Edges	Fraud%*
Training	Static	56.7M	153.9M	
	Snapshot	56.7M	274.5M	2.70%
	TD	66.4M	332.0M	
Testing	Static	9.63M	18.42M	
	Snapshot	9.63M	30.39M	2.30%
	TD	10.28M	32.88M	

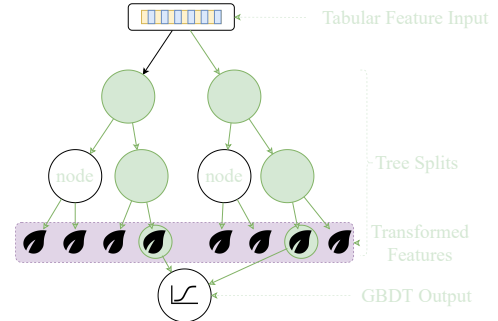


Figure 3: GBDT as Feature Encoder.

of multiple hops, we also use one convolution layer for RT Net, to refrain from data leakage.

4 FRAUD DETECTION EXPERIMENT

In this section, we perform an evaluation of BRIGHT's performance in the fraud detection task.

4.1 Dataset and Preprocessing

We conduct experiments on real-world transaction records sampled from eBay marketplace to evaluate the proposed framework. Our dataset contains 7 million labeled transactions.⁷ Downsampling is

⁵In consideration of both inference efficiency and the prevention of data leakage, a single layer is chosen for RT Net. (1) Obviously, one convolutional layer provides lower inference latency. (2) Currently, only one-hop neighbours to the target transactions are linked by unidirectional edges, the other neighboring nodes are connected by bidirectional edges. Therefore, if there are more than one convolutional layers, we would have a data leakage problem, where the future information could be used to predict nodes in the past.

⁶<https://neo4j.com/developer/cypher/>.

⁷Note that in our experiments, only these 7 million transaction records have labels, the graphs in Tab. 4 contain more transaction nodes yet without labels (reference

conducted on legitimate transactions due to the label imbalance, as in [17]. The sampling ratio for fraud transactions is kept at ar. 2.5%, which is empirically determined, as in other use cases at eBay. Entities in $\mathcal{G}$ include shipping addresses, device IDs, phones, payment tokens, IPs, and buyer accounts.

The data set is divided by the creation time of the target transactions into training and testing datasets. The first 80% target transactions are used as a training dataset, and the last 20% transactions are used for testing. The validation set for early stopping check is 10%, randomly chosen from the training dataset. The statistics for the graph are illustrated in the Tab. 4.

4.2 Baselines

LightGBM (LGB) [11] is chosen as the baseline. There are two baseline models in our experiments built upon LightGBM. One is fully trained, namely, LGB (Bench), with 10k + trees, and was early-stopped after 32 iterations if the validation metrics have not improved. The other is a light version with 512 trees. The light model, namely the LGB (FE), is also utilized as a feature encoder (FE) for the LNN input.

4.3 Feature Encoder

Gradient Boosting Decision Tree (GBDT) models such as LightGBM [11] dominate many tabular dataset applications [7], compared to existing popular deep model architectures designed specifically for tabular dataset, such as AutoInt [21] and TabNet [1]. For fraud detection on payment transactions, where most datasets are tabular, boosting models are widely used [3, 31]. For simplicity, a pre-trained GBDT model is used as the feature encoder for tabular features. Similarly to Logistic Regression models on top of Decision Trees [9], the values of the chosen leaf nodes are used as encoded embeddings from the feature encoder, as illustrated in Fig. 3.

4.4 Experiment Setup

The transaction records sampled as described in Sec. 4.1 are transformed into graphs using our method proposed in Sec. 3.2.

(Data Loader.) The mini-batch strategy is used in GNN learning. In each mini-batch, there is only one target partition. The maximum number of historical time snapshots ($\mathcal{T}$) in the target partition is set to 30. The size of the mini-batch depends on the merged community⁸ node size, which is close to 32k. The transformed TD graph is used as input to the LNN model. $\mathcal{G}_t^{Batch}$ and $\mathcal{G}_t^{RT}$ are fed separately for LNN models (Batch Net and RT Net, respectively).

(Feature Encoding.) The features of txn_t^{ref} and txn_t^{tgt} are first encoded by LGB (FE). The embeddings X^t generated by LGB (FE)

transactions). It is common that not every transaction has a label, which could be due to delay in third-party service, as we elaborated in [17].

⁸Similar to ClusterGCN [4], graph clustering structure is used in the BRIGHT training process. Historical neighboring reference transactions are 2 hops away from the target transactions. For the connected components, whose node sizes are larger than 32k, they are cut into smaller communities through Louvain [2]. Before training, small communities are merged in the same target partition. The node sizes of the merged communities are close to 32k. The batch size is chosen by experience to balance disk IO and GPU memory. The impact of batch size in both efficiency and fraud detection performance could be investigated in our future work. Different batches are not overlapping in terms of target transactions that are predicted. But the features and nodes used for each prediction could be overlapping if they are used/linked in the neighborhood of the target transactions.

Table 5: LNN Performances in Fraud Detection (Test Set).
LGB: LightGBM, LNN: Lambda Neural Network.

Model	ROC AUC	Average Precision
LGB (FE)	0.9255±0.0003	0.3984±0.0007
LGB (Bench)	0.9247±0.0005	0.4201±0.0013
BRIGHT-LNN (GAT)	0.9286±0.0005	0.4394±0.0009
BRIGHT-LNN (GCN)	0.9276±0.0016	0.4420±0.0035

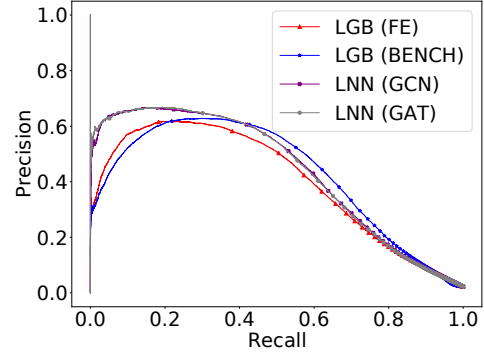


Figure 4: Precision-Recall Curves.

are then fed to LNN, together with $\mathcal{G}_t^{Batch}$ and $\mathcal{G}_t^{RT}$. The features of $entity_t^{tgt}$ are set to zero. The vector size of the transaction embeddings and entity embeddings is 512 (equal to the number of trees). The remaining parameters to train GBDT are default values.

(Model Architecture.) As in DeepGCN [14], we choose GCN [12] and GAT [23] for Batch Net, and GCN for RT Net.

(Hyperparameters.) We grid search for the number of GNN layers for Batch Net and the number of hidden units for both Batch Net and RT Net. The numbers of GNN layers are chosen from {4, 6, 8, 16}. The numbers of hidden units are chosen from {256, 512}. The learning rate is set to 0.001 for all experiments. The maximal epochs for training are 128. Regarding early stopping, we terminate the training if the loss over the validation dataset does not decrease within 16 epochs. For each group of hyperparameters, we execute three runs and report the average results in Tab. 5.

4.5 Quantitative Results on Fraud Detection

We report the results of Average Precision (AP) and Area Under the Receiver Operating Characteristic Curve (ROC AUC) on the testing data in Tab. 5. As can be seen, LNN outperforms LGB (FE) in terms of both ROC AUC and AP, by aggregating features through graph topology. LNN (GCN) has an average precision of 44.2%. Compared to LGB (Bench), which is the state-of-the-art model for tabular data, LNN (GCN) achieves an improvement of 2.2%. We also plot the precision-recall curves in Fig. 4, and the curves show that LNN models can capture more fraud transactions with higher precision if we prioritize precision over recall. In our previous work [17], we have discussed that in fraud detection tasks, we generally maintain a balance between precision and recall, while favoring precision over recall. This is because we try to reduce the workload of the

business unit in examining false positives while trying to catch as many frauds as possible.

5 LAMBDA NEURAL NETWORK ARCHITECTURE EFFICIENCY

Optimizing inference efficiency is another goal in our BRIGHT framework, which aims to minimize neighbor queries during GNN inference. In this evaluation, we compare LGB (Bench), end-to-end LNN (GCN), and RT Net.

5.1 Experiment Setup

For LNN (GCN) and RT Net, we use both CPU and GPU devices. In each test, a target transaction is evaluated and the system evaluates its risk of fraud. We report the real-time inference latency, which is decoupled into three stages – *graph query*, *feature collection*, and *model inference*.

(Graph Query.) For GNN inference, neighboring reference transactions are required for graph construction and feature collection. Theoretically, key-value databases could be used to support query services for neighbor lookup. However, they are not good choices due to the complexity of the service implantation. Graph databases are natural choices as they provide a standard graph query language. In our experiments, Neo4j Community Edition⁹ is used as our graph query service.

We set up the database for one target partition, which is randomly selected from the test set. There are 0.7 million nodes and 3.8 million edges loaded into the graph DB. The default configuration is used for the Neo4j server. For LGB (Bench) and RT Net, there is no graph query test. LGB (Bench) does not need relational data for inference. In RT Net, the 1-hop entity IDs for embedding collection are in the request to predict the label of target transaction.

(Feature Collection.) Both raw features and entity embeddings are stored in a key-value database in our experiments. For simplicity, Redis¹⁰ is chosen as our key-value database. For these 0.7 million records, we store their raw features and embeddings on the Redis server with the default configuration.

(Model Inference.) LGB (Bench) inference is conducted on CPU only. LNN (E2E) inference is an end-to-end full GNN inference, where we do not separate the Batch Net and RT Net. LNN (E2E) is equivalent to a traditional GNN inference workflow. The graph convolutional layers chosen for LNN is GCN. For the tests of RT Net inference, there is no Batch Net inference. Entity embeddings from batch inference are obtained from the key-value store. Both CUDA and CPU devices are tested for LNN and RT Net inference. Both models are implemented in PyTorch. For simplicity, there is no further optimization of the inference for the LNN models and RT Net models. We randomly choose 10k payment transactions for the latency test, recorded in milliseconds. We use one machine for the database server and inference client. Regarding hardware, the machine is equipped with 32 Intel Xeon Gold 6230 CPUs, 64 GB memory, and one Nvidia Tesla V100 GPU.

5.2 Experimental Results

As can be seen in Tab. 6, we compare LNN (E2E) and RT Net. The end-to-end inference 99% latency (99th percentile latency, a.k.a. P99) is significantly reduced from 251.35 ms to 72.57 ms using RT Net, that is, 71.13% latency saving on CUDA devices. In CPU devices, the latency saving is even greater (76.67%). The reason for the performance improvement is that the graph query and Batch Net inference are omitted in RT Net. Compared to LGB, although RT Net introduces a more complex model architecture, only around 1.6% overhead of P99 inference latency is introduced in the inference stage. Note that our latency numbers (<100ms) resonate the results reported by previous benchmarking papers in the GNN inference optimization [30, 32].

5.3 Discussion on Extended Graph Inference

In our previous experiments, we only choose two-hop neighbors (i.e., entities and their reference transactions) for the target transaction. As readers might suspect, *what if neighbors of more hops are considered?* To answer this question, we perform experiments to evaluate inference latency with 4-hop and 6-hop neighbors. We take a daily partition with up to 6-hop neighbors (i.e., extended graph), which has 10.5 million nodes and 41.3 million edges. As illustrated in the Tab. 7, as the neighbor hops increase, our RT-Net architecture achieves larger inference speedups. For example, LNN on CPU devices for the 6-hop neighbor graph is 1335.98 ms, while RT Net only costs 58.44 ms — a 22.86× improvement.

5.4 Discussion on Using BRIGHT in Other Production Scenarios

We can use the Two-Stage Directed Graph in scenarios such as fraud detection for payment transaction, where we require constant update of the transaction snapshots (the frequency should be less than one day). In this use case, the current design in BRIGHT would not be optimal, as Batch Net embeddings are not updated frequently enough due to the low update frequency of data warehouse. We can mitigate this problem by the integration with graph database in data warehousing.

6 RELATED WORK

We discuss previous work that is relevant to BRIGHT.

(Graph Neural Network.) GNN [8, 12, 22] has become increasingly popular in learning from graphs. Through message passing, aggregation and self-attention, it has a powerful capacity to grasp the graph structure, as well as the complex relations between nodes. Recent work on GNN has also focused on heterogeneous graphs [10, 15], where nodes and edges can be of various types.

(Modelling Dynamics with GNN.) Dynamic graphs can also be represented as a sequence of time events. There are several works along this line and mostly on homogeneous graphs. Temporal Graph Networks (TGN) [19] applied memory modules and graph-based operators. The TGN framework is computationally efficient based on event update. Asynchronous Propagation Attention Network (APAN) [26] adopted a temporal encoding similar to TGN and decoupled graph computation and inference. However, for TGN, only a few neighbors are accessible by the graph module due to memory constraints. One typical work is DySAT [20] which applies

⁹<https://neo4j.com/>

¹⁰<https://redis.io/>

Table 6: Inference Latency (recorded in millisecond). Numbers in brackets refer to speedups compared to LNN (E2E). In LNN (E2E) there is no separation of the Batch Net and RT Net. LGB: LightGBM, LNN: Lambda Neural Network, RT: Real-Time.

Model	LGB		LNN (E2E, CUDA)		LNN (E2E, CPU)		BRIGHT-RT Net (CUDA)		BRIGHT-RT Net (CPU)	
Step	Avg	P99	Avg	P99	Avg	P99	Avg	P99	Avg	P99
Graph Query	-	-	114.42	177.60	114.42	177.60	-	-	-	-
Feature Collection	0.06	0.13	0.08	0.20	0.08	0.20	0.06 (1.33×)	0.11 (1.82×)	0.06 (1.33×)	0.11 (1.82×)
Feature Encoding	-	-	51.88	72.16	51.88	72.16	51.70 (1.00×)	71.93 (1.00×)	51.70 (1.00×)	71.93 (1.00×)
Model Inference	52.12	71.33	15.26	29.86	150.98	365.83	1.94 (7.87×)	6.22 (4.80×)	20.74 (7.28×)	63.81 (5.73×)
Total	52.18	71.40	181.64	251.35	317.36	532.69	53.70 (3.38×)	72.57 (7.34×)	72.44 (4.38×)	124.26 (4.29×)

Table 7: Model Inference Latency in Extended Graph (recorded in milliseconds). Numbers in brackets refer to speedups compared to LNN (E2E). In LNN (E2E) there is no separation of the Batch Net and RT Net. LGB: LightGBM, LNN: Lambda Neural Network, RT: Real-Time.

Model		LNN (E2E)		BRIGHT-RT Net	
Device	Hop	Avg	P99	Avg	P99
CUDA	2	15.26	29.86	1.94 (7.87×)	6.22 (4.80×)
	4	14.65	30.66	1.84 (7.96×)	5.90 (5.20×)
	6	16.14	57.03	2.01 (8.01×)	6.46 (8.83×)
CPU	2	150.98	365.83	20.74 (7.28×)	63.81 (5.73×)
	4	171.69	1044.73	16.21 (10.59×)	49.86 (20.95×)
	6	216.36	1335.98	19.00 (11.39×)	58.44 (22.86×)

self-attention networks to learn low-dimensional embeddings of nodes in a dynamic homogeneous graph. One notable difference in our setting is that we need to distinguish between two types of entities (historical and future), while DySAT assumes that all entities can be added or removed in the graph at any time point.

(Dynamic GNN on Fraud Detection.) As discussed on Sec. 2, we have done previous work on flagging frauds in a dynamic setting (DHGReg [18] and DyHGN [16]). However, these approaches are not efficient in online deployment due to three issues that we analyzed in Sec. 2. In BRIGHT, we design modules to address these issues so that GNN is deployable in real-time.

(Two-Tower Models.) We have surveyed the literature on two-tower / stage models that attempt to incorporate batch and streaming processing in a Lambda architecture. In [5, 28], a two-tower model is proposed to pair users and items in the video recommendation system. One tower is for user embedding generation, and the other tower is adopted for item embedding generation. The Dual Augmented Two-tower Model (DAT) [29] customizes an augmented vector for each query and item to tackle imbalanced category data. Attention layers are utilized in [24] to improve the recommendation of warm- and cold-start items in online video services. Cross-Batch Negative Sampling (CBNS) techniques [25] are used to increase training of the two-tower model. Mixed Negative Sampling (MNS) [27] uses a mix of batch and uniform sample strategies to reduce selection bias during training for the app recommendation system. These applications focus on deep models on recommendation systems, with user embeddings and item embeddings precalculated. We setup a similar framework in BRIGHT, the neighbor embeddings

are learned by a Batch Net and stored in a key-value store, which reduces the cost in online GNN deployment.

(Model Inference Acceleration on GNN.) Recently, people started looking at the efficiency of online GNN inference. [32] applies channel pruning to speed up real-time GNN inference, by reducing Multiplication-and-ACcumulation (MAC) operations. They have also explored precomputation and storing of node representations to boost performance. GLNN [30] applied knowledge distillation (KD) to teach multi-layer perceptrons (MLPs) from trained GNN models. The performance of GLNN is competitive. However, the graph neighbor information is not utilized in inference. Our work focuses on relation modeling, so our model is still a vanilla GNN model. In BRIGHT, during real-time inference, neighbor information is still accessed from our precalculated entity embeddings. Our BRIGHT framework is among the first to scale dynamic GNN into an industrial dataset (>60 million nodes and >160 million edges as we show in Tab. 6 for static graph) and has provided an industrial benchmark in system deployment.

7 CONCLUSION

In this paper, we explore how to apply GNN for fraud detection. We present a novel BRIGHT framework for efficient end-to-end learning and real-time inference without the leakage of future information. We designed a graph transformation approach to avoid future information leakage. Furthermore, a Lambda neural network architecture was proposed to support both batch and real-time processing. Our empirical study showed that BRIGHT outperformed baselines by 2% w.r.t. average precision and significantly reduced inference latency by 7.8× on average.

ACKNOWLEDGMENTS

CZ and the DS3Lab gratefully acknowledge the support from the Swiss State Secretariat for Education, Research and Innovation (SERI) under contract number MB22.00036 (for European Research Council (ERC) Starting Grant TRIDENT 101042665), the Swiss National Science Foundation (Project Number 200021_184628, and 197485), Innosuisse/SNF BRIDGE Discovery (Project Number 40B2-0_187132), European Union Horizon 2020 Research and Innovation Programme (DAPHNE, 957407), Botnar Research Centre for Child Health, Swiss Data Science Center, Alibaba, Cisco, eBay, Google Focused Research Awards, Kuaishou Inc., Oracle Labs, Zurich Insurance, and the Department of Computer Science at ETH Zurich.

REFERENCES

- [1] Sercan O Arık and Tomas Pfister. 2021. Tabnet: Attentive interpretable tabular learning. In *AAAI*, Vol. 35. 6679–6687.
- [2] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (oct 2008), P10008. <https://doi.org/10.1088/1742-5468/2008/10/p10008>
- [3] Shaosheng Cao, XinXing Yang, Cen Chen, Jun Zhou, Xiaolong Li, and Yuan Qi. 2019. Titant: Online real-time transaction fraud detection in ant financial. *arXiv preprint arXiv:1906.07407* (2019).
- [4] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM. <https://doi.org/10.1145/3292500.3330925>
- [5] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [6] Rishabh Goel, Seyed Mehran Kazemi, Marcus Brubaker, and Pascal Poupart. 2020. Diachronic embedding for temporal knowledge graph completion. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 3988–3995.
- [7] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems* 34 (2021).
- [8] William L. Hamilton, Z. Ying, and J. Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NIPS*.
- [9] Xinran He, Junfeng Pan, Ou Jin, Tianbing Xu, Bo Liu, Tao Xu, Yanxin Shi, Antoine Atallah, Ralf Herbrich, Stuart Bowers, et al. 2014. Practical lessons from predicting clicks on ads at facebook. In *Proceedings of the eighth international workshop on data mining for online advertising*. 1–9.
- [10] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*. 2704–2710.
- [11] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017), 3146–3154.
- [12] Thomas Kipf and M. Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *ArXiv abs/1609.02907* (2017).
- [13] Ivan Launders and Simon Polovina. 2013. Chapter 13 - A Semantic Approach to Security Policy Reasoning. In *Strategic Intelligence Management*, Babak Akhgar and Simeon Yates (Eds.). Butterworth-Heinemann, 150–166. <https://doi.org/10.1016/B978-0-12-407191-9.00013-2>
- [14] Guohao Li, Matthias Müller, Ali Thabet, and Bernard Ghanem. 2019. DeepGCNs: Can GCNs Go as Deep as CNNs? *arXiv:1904.03751 [cs.CV]*
- [15] Qingsong Lv, Ming Ding, Qiang Liu, Yuxiang Chen, Wenzheng Feng, Siming He, Chang Zhou, Jianguo Jiang, Yuxiao Dong, and Jie Tang. 2021. Are we really making much progress? Revisiting, benchmarking and refining heterogeneous graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1150–1160.
- [16] Susie Xi Rao, Clémence Lanfranchi, Shuai Zhang, Zhichao Han, Zitao Zhang, Wei Min, Mo Cheng, Yanan Shan, Yang Zhao, and Ce Zhang. 2022. Modelling graph dynamics in fraud detection with “Attention”. *arXiv:2204.10614 [cs.LG]*
- [17] Susie Xi Rao, Shuai Zhang, Zhichao Han, Zitao Zhang, Wei Min, Zhiyao Chen, Yanan Shan, Yang Zhao, and Ce Zhang. 2021. xFraud: explainable fraud transaction detection. *Proceedings of the VLDB Endowment* 15, 3 (2021), 427–436.
- [18] Susie Xi Rao, Shuai Zhang, Zhichao Han, Zitao Zhang, Wei Min, Mo Cheng, Yanan Shan, Yang Zhao, and Ce Zhang. 2020. Suspicious Massive Registration Detection via Dynamic Heterogeneous Graph Neural Networks. *arXiv:2012.10831 [cs.LG]*
- [19] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637* (2020).
- [20] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. 2020. Dysat: Deep neural representation learning on dynamic graphs via self-attention networks. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 519–527.
- [21] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. AutoInt: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 1161–1170.
- [22] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. *ArXiv abs/1706.03762* (2017).
- [23] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [24] Jianling Wang, Ainur Yessenalina, and Alireza Roshan-Ghias. 2021. Exploring Heterogeneous Metadata for Video Recommendation with Two-tower Model. *arXiv preprint arXiv:2109.11059* (2021).
- [25] Jinpeng Wang, Jieming Zhu, and Xiuqiang He. 2021. Cross-Batch Negative Sampling for Training Two-Tower Recommenders. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1632–1636.
- [26] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, and et al. 2021. APAN: Asynchronous Propagation Attention Network for Real-time Temporal Graph Embedding. *Proceedings of the 2021 International Conference on Management of Data* (Jun 2021). <https://doi.org/10.1145/3448016.3457564>
- [27] Ji Yang, Xinyang Yi, Derek Zhiyuan Cheng, Lichan Hong, Yang Li, Simon Xiaoming Wang, Taibai Xu, and Ed H Chi. 2020. Mixed negative sampling for learning two-tower neural networks in recommendations. In *Companion Proceedings of the Web Conference 2020*. 441–447.
- [28] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Ajit Kumthekar, Zhe Zhao, Li Wei, and Ed Chi (Eds.). 2019. *Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations*.
- [29] Yantao Yu, Weipeng Wang, Zhoutian Feng, and Daiyue Xue. 2021. A Dual Augmented Two-tower Model for Online Large-scale Recommendation. (2021).
- [30] Shichang Zhang, Yozen Liu, Yizhou Sun, and Neil Shah. 2021. Graph-less neural networks: Teaching old mlps new tricks via distillation. *arXiv preprint arXiv:2110.08727* (2021).
- [31] Hao Zhou, Hong-feng Chai, and Mao-lin Qiu. 2018. Fraud detection within bankcard enrollment on mobile device based payment using machine learning. *Frontiers of Information Technology & Electronic Engineering* 19, 12 (2018), 1537–1545.
- [32] Hongkuan Zhou, Ajitesh Srivastava, Hanqing Zeng, Rajgopal Kannan, and Viktor Prasanna. 2021. Accelerating large scale real-time GNN inference using channel pruning. *arXiv preprint arXiv:2105.04528* (2021).